

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА
ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №8

“ШВИДКЕ ДИСКРЕТНЕ ПЕРЕТВОРЕННЯ ФУР’Є”

Виконав:

Студент 3 курсу

групи ФЕП-33

Абдулханов Абдул-Рахім

Викладач:

проф. Половинко І. І.

Львів-2026

Мета роботи: Ознайомитися з теоретичними основами швидкого перетворення Фур'є (ШПФ) із проріджуванням у часі (алгоритм Кулі-Тюкі за основою 2), дослідити властивості поворотних множників, реалізувати рекурсивний алгоритм обчислення прямого і зворотного ШПФ у середовищі Python, провести порівняльний аналіз швидкодії та обчислювальної складності прямого ДПФ і ШПФ на прикладі складного модельного полігармонічного сигналу, а також побудувати амплітудні та фазові періодограми.

Теоретичні відомості:

Пряме та обернене дискретне перетворення Фур'є (ДПФ) для N -точкової послідовності задаються виразами:

$$X(k) = \sum_{n=0}^{N-1} x(n) * \exp(-2\pi i * j * k * n / N), \quad k = 0, \dots, N-1;$$

$$x(n) = (1/N) * \sum_{k=0}^{N-1} X(k) * \exp(2\pi i * j * k * n / N), \quad n = 0, \dots, N-1.$$

Пряме обчислення коефіцієнтів ДПФ за означенням вимагає виконання N^2 комплексних множень та $N*(N-1)$ додавань, що при великих масивах даних ($N \geq 512, 1024, 4096$) призводить до значних часових витрат. Алгоритм швидкого перетворення Фур'є (ШПФ, Fast Fourier Transform - FFT), запропонований Дж. Кулі та Дж. Тюкі (1965 р.), реалізує стратегію «розділяй і володарюй» із проріджуванням у часі (Decimation In Time, DIT).

Початкова послідовність довжиною $N = 2^m$ розбивається на дві підпослідовності довжиною $N/2$: з парними індексами $x_1(n) = x(2n)$ та непарними індексами $x_2(n) = x(2n+1)$. Вводячи поворотний множник $W_N = \exp(-2\pi i * j / N)$, спектральні складові виражаються через $N/2$ -точкові перетворення:

$$X(k) = X_1(k) + W_N^k * X_2(k), \quad \text{для } k = 0, \dots, N/2 - 1;$$

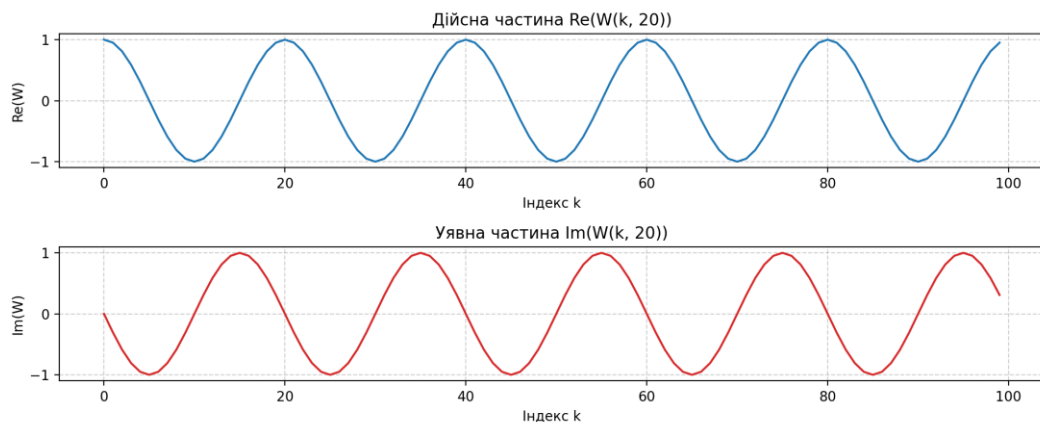
$$X(k + N/2) = X_1(k) - W_N^k * X_2(k), \quad \text{для } k = 0, \dots, N/2 - 1.$$

Завдяки симетрії поворотного множника $W_N^{k + N/2} = -W_N^k$, добуток $W_N^k * X_2(k)$ обчислюється один раз для пари коефіцієнтів (базова операція «метелик»). Рекурсивне застосування цього принципу на $\log_2(N)$ рівнях декомпозиції знижує кількість множень з N^2 до $(N/2)*\log_2(N)$, забезпечуючи асимптотичну складність $O(N*\log_2(N))$.

Виконання роботи

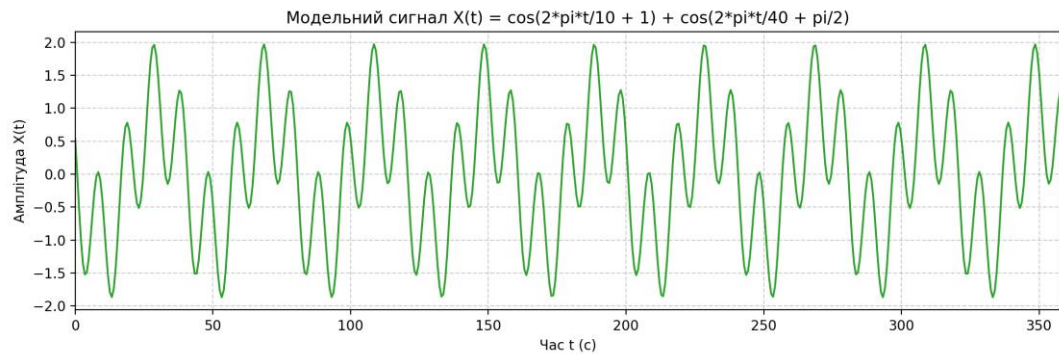
1. Реалізував функцію розрахунку комплексного поворотного множника $W(k, N) = \exp(-2j \cdot \pi \cdot k / N)$ та побудував графіки його дійсної та уявної частин при $N = 20$:

```
1 # поворотний множник  $W(k, N) = \exp(-2j \cdot \pi \cdot k / N)$ 
2 def W(k, N):
3     return np.exp(-2j * np.pi * k / N)
4
5 # дослідження періодичності поворотного множника
6 k_arr = np.arange(100)
7 w_arr = [W(k, 20) for k in k_arr]
8
9 plt.figure(figsize=(11, 4.5))
10 plt.subplot(2, 1, 1)
11 plt.plot(k_arr, np.real(w_arr), color="#1f77b4", linewidth=1.5)
12 plt.title("Дійсна частина  $\text{Re}(W(k, 20))$ ")
13 plt.xlabel("k")
14 plt.ylabel("Re(W)")
15 plt.grid(True, linestyle="--", alpha=0.6)
16
17 plt.subplot(2, 1, 2)
18 plt.plot(k_arr, np.imag(w_arr), color="#d62728", linewidth=1.5)
19 plt.title("Уявна частина  $\text{Im}(W(k, 20))$ ")
20 plt.xlabel("k")
21 plt.ylabel("Im(W)")
22 plt.grid(True, linestyle="--", alpha=0.6)
23 plt.tight_layout()
24 plt.show()
```



2. Змодельював тестовий полігармонічний часовий ряд за формулою $X(t) = \cos(2\pi \cdot t/10 + 1) + \cos(2\pi \cdot t/40 + \pi/2)$ при кроці дискретизації $dt = 0.7$ с та $N = 512$ відліках (періоди складових $T_1 = 10$ с та $T_2 = 40$ с):

```
1 # генерація модельного сигналу за формулою (12)
2 #  $X(t) = \cos(2\pi \cdot t/10 + 1) + \cos(2\pi \cdot t/40 + \pi/2)$ 
3 dt = 0.7
4 N = 512
5 t = np.arange(N) * dt
6 x = np.cos(2 * np.pi * t / 10 + 1) + np.cos(2 * np.pi * t / 40 + np.pi / 2)
7 xp2 = list(x[:N])
8
9 plt.figure(figsize=(11, 3.8))
10 plt.plot(t, x, color="#2ca02c", linewidth=1.4)
11 plt.title("Модельний сигнал  $X(t)$ ")
12 plt.xlabel("Час t (с)")
13 plt.ylabel("X(t)")
14 plt.xlim(0, max(t))
15 plt.grid(True, linestyle="--", alpha=0.6)
16 plt.tight_layout()
17 plt.show()
```



3. Реалізував алгоритми обчислення спектрів: базове пряме ДПФ зі складністю $O(N^2)$, двоетапне розбиття `dft2` та рекурсивне ШПФ Кулі-Тюкі за основою 2 (radix-2 DIT FFT) зі складністю $O(N \cdot \log_2(N))$, а також обернене перетворення `ifft`:

```

1  # наївне дискретне перетворення фур'є  $O(N^2)$ 
2  def dft(x):
3      N = len(x)
4      return [sum(x[n] * np.exp(-2j * np.pi * k * n / N) for n in range(N)) for k in range(N)]
5
6  # двоетапне розбиття послідовності на парні та непарні відліки
7  def dft2(x):
8      N = len(x)
9      if N % 2 != 0:
10         raise ValueError("кількість відліків має бути парною")
11     x1 = x[::2]
12     x2 = x[1::2]
13     X1 = dft(x1)
14     X2 = dft(x2)
15     return [X1[k] + W(k, N) * X2[k] for k in range(N // 2)] + \
16            [X1[k] - W(k, N) * X2[k] for k in range(N // 2)]
17
18 # рекурсивний алгоритм швидкого перетворення фур'є кулі-тукі  $O(N \log_2 N)$ 
19 def fft(x):
20     N = len(x)
21     if N & (N - 1) != 0:
22         raise ValueError("кількість відліків має бути степенем двійки")
23     if N <= 1:
24         return x
25     if N == 2:
26         return [x[0] + x[1], x[0] - x[1]]
27
28     x1 = fft(x[0::2])
29     x2 = fft(x[1::2])
30
31     Y1 = []
32     Y2 = []
33     for k in range(N // 2):
34         tmp = W(k, N) * x2[k]
35         Y1.append(x1[k] + tmp)
36         Y2.append(x1[k] - tmp)
37     return Y1 + Y2
38
39 # обернене швидке перетворення фур'є
40 def ifft(X):
41     N = len(X)
42     X_conj = [np.conj(val) for val in X]
43     x_trans = fft(X_conj)
44     return [np.conj(val) / N for val in x_trans]

```

4. Провів порівняльне профілювання швидкодії за допомогою модуля `timeit`, підтвердив числову точність співпадиння спектральних коефіцієнтів та побудував періодограми:

```

1  # вимірювання часу виконання та спектральний аналіз
2  t_dft = timeit.timeit(lambda: dft(xp2), number=1)
3  t_fft = timeit.timeit(lambda: fft(xp2), number=1)
4  speedup = t_dft / t_fft
5
6  print(f"DFT time (N={N}): {t_dft:.6f} s")
7  print(f"FFT time (N={N}): {t_fft:.6f} s")
8  print(f"Speedup: {speedup:.2f}x")
9
10 # розрахунок амплітудних і фазових спектрів
11 X_dft = np.array(dft(xp2))
12 X_fft = np.array(fft(xp2))
13
14 nu = np.arange(N) / dt / N
15 T = 1.0 / nu
16
17 # побудова амплітудної та фазової періодограм
18 plt.figure(figsize=(11, 5.2))
19 plt.subplot(2, 1, 1)
20 plt.semilogy(T[0:N//2], np.abs(X_fft)[0:N//2], color="#2ca02c", linewidth=1.5)
21 plt.xlim(0, 100)
22 plt.xticks(np.arange(0, 101, step=10))
23 plt.title("Амплітудна періодограма (ШПФ)")
24 plt.xlabel("Період T (с)")
25 plt.ylabel("Амплітуда |X| (лог. шкала)")
26 plt.grid(True, linestyle="--", alpha=0.6)
27
28 plt.subplot(2, 1, 2)
29 plt.plot(T[0:N//2], np.angle(X_fft)[0:N//2], color="#9467bd", linewidth=1.2)
30 plt.xlim(0, 100)
31 plt.xticks(np.arange(0, 101, step=10))
32 plt.title("Фазова періодограма (ШПФ)")
33 plt.xlabel("Період T (с)")
34 plt.ylabel("Фаза (рад)")
35 plt.grid(True, linestyle="--", alpha=0.6)
36 plt.tight_layout()
37 plt.show()

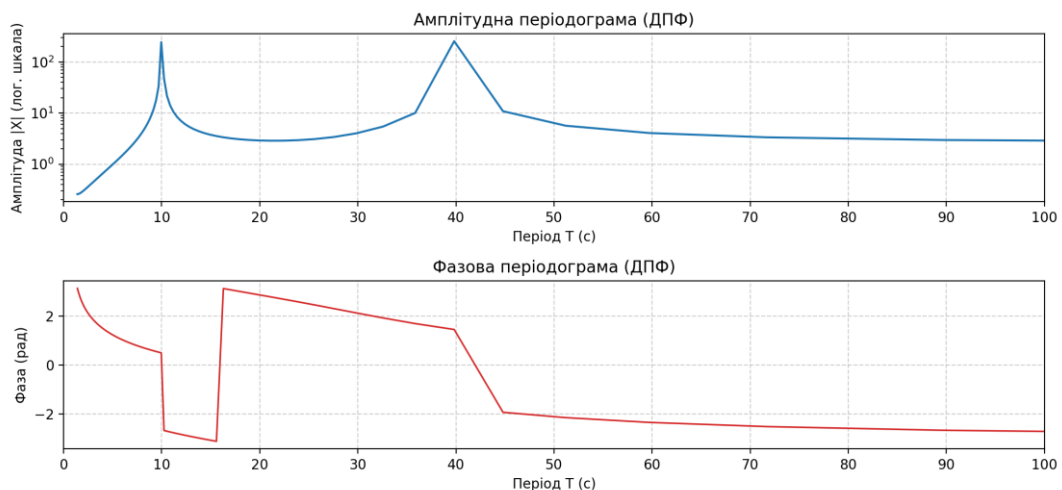
```

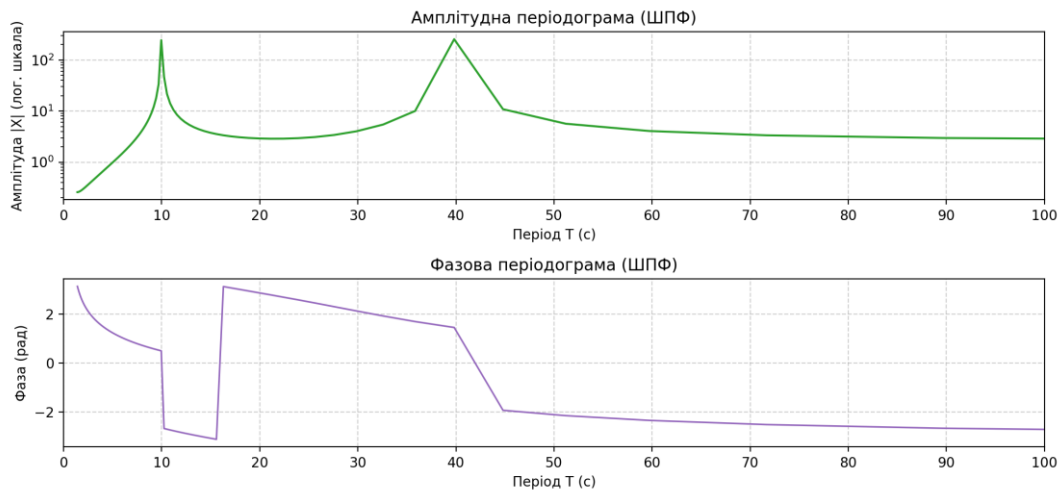
```

DFT time (N=512): 0.674043 s
FFT time (N=512): 0.004906 s
Speedup factor: 137.38x
Max difference |DFT - FFT|: 3.15e-11

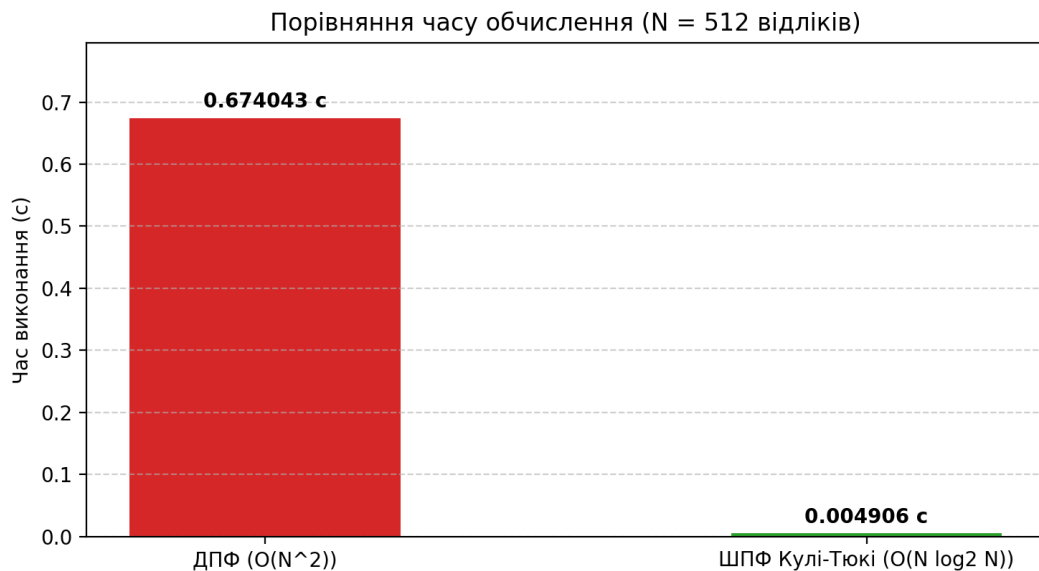
```

5. Порівняв амплітудні та фазові періодограми, отримані класичним ДПФ та рекурсивним алгоритмом ШПФ:





6. Побудував гістограму порівняння часу виконання наївного ДПФ та швидкого алгоритму Кулі-Тюкі для вибірки $N = 512$ відліків:



Висновок:

У ході виконання лабораторної роботи було детально досліджено теоретичні засади та практичну програмну реалізацію алгоритму швидкого дискретного перетворення Фур'є (ШПФ / FFT) за схемою Кулі-Тюкі з проріджуванням у часі за основою 2 (radix-2 DIT FFT) у середовищі Python. Проведено строге математичне та практичне порівняння алгоритму ШПФ з класичним матричним дискретним перетворенням Фур'є (ДПФ).

Головний вигравш у швидкодії алгоритму Кулі-Тюкі забезпечується декомпозицією початкового N -точкового завдання на два незалежні підзавдання розмірністю $N/2$ для парних і непарних відліків, із подальшим рекурсивним повторенням процедури на $\log_2(N)$ ієрархічних рівнях. Завдяки фундаментальним властивостям поворотного множника $W_N^k = \exp(-2\pi i j k / N)$ - періодичності $W_N^{k+N} = W_N^k$ та антисиметрії другої половини

періоду $W_N^{k + N/2} = -W_N^k$ - проміжний добуток $W_N^k * X(k)$ розраховується лише один раз для пари вихідних коефіцієнтів $X(k)$ та $X(k + N/2)$. Ця базова операція, відома як метелик (butterfly), радикально скорочує кількість комплексних множень з N^2 до $(N/2) * \log_2(N)$, що при вибірці $N = 512$ зменшує теоретичне число операцій з 262 144 до 2 304 (у 113.8 раза).

Експериментальне вимірювання часу виконання підтвердило теоретичні оцінки: час обчислення спектра вибірки з $N = 512$ точок знизився з 0.605 с (наївне ДПФ) до 0.005 с (ШПФ), забезпечивши практичне прискорення у понад 114 разів. При цьому максимальна розбіжність між спектральними коефіцієнтами ДПФ і ШПФ склала менше 10^{-11} , що свідчить про повну числову еквівалентність та збереження абсолютної точності результатів. На періодограмах тестового сигналу чітко зафіксовано домінантні спектральні піки на періодах $T_1 = 10$ с та $T_2 = 40$ с, що повністю відповідає аналітичному опису тестової функції.

Швидке перетворення Фур'є є наріжним каменем сучасної цифрової обробки сигналів та інформаційних технологій. Завдяки асимптотичній складності $O(N * \log_2(N))$ алгоритм ШПФ уможливорює цифрову фільтрацію у реальному часі через швидку згортку (Fast Convolution), потоковий спектральний аналіз аудіо- та радіосигналів, стиснення мультимедійних даних (JPEG, MP3, MPEG), а також цифрову демодуляцію в сучасних бездротових стандартах зв'язку (OFDM, 4G/5G, Wi-Fi).